

template engine and Werkzeug WSGI. The remaining decisions are left to you, as the developer. You can choose different extensions depending on your needs and interests. Let's create a simple Web application using Flask.

Flask installation

As a prerequisite, you need Python 2.6 or higher to get started, and Pip, which is a tool to install and manage Python packages. Pip makes it easy for users to install and uninstall Python packages in a single command. For example:

```
$pip install PackageName
$pip uninstall PackageName
```

You have two ways to install Flask on your machine. You can either choose a virtual environment that allows you to work in an isolated environment with a separate copy of Python, and without worrying about affecting other projects if anything goes wrong, or you can go for a direct installation of Python and Flask on your native machine. I have explained both the methods here.

Let us first try the virtual environment using Pip. Ubuntu users can try it directly without Pip.

```
$sudo pip install virtualenv
```

Ubuntu users should use:

```
$sudo apt-get install python-virtualenv
```

...to begin the virtual environment, and activate it by issuing the following command:

```
$ virtualenv venv
$. venv/bin/activate
```

You can deactivate *virtualenv* as follows:

```
$deactivate
```

Once you have activated *virtualenv*, you are ready to install Flask in the virtual environment. Do so by using Python package management.

```
$pip install Flask
$pip install flask-script
```

You can install Flask directly in the system by avoiding the virtual environment (sometimes Flask comes with Python):

```
$ pip install Flask
$ pip install flask-script
```

Once successfully installed, you can go ahead to create a simple Web application.

Our first app

Let's create a directory for our application to start off with Flask, as follows:

```
$ mkdir myapp
$ cd myapp

File structure:
myapp
  __init__.py
  manage.py
  templates/
    about.html
    base.html
    index.html
    posts.html
  static/
css/
  style.css
```

Create the directories and files which are listed above. *__init__.py* will create an object for application using Flask and will be ready to run the app.

manage.py will handle all URL routing properties and content. The *templates* directory contains all HTML pages needed for the applications and *static* contains all static resources like the style sheet, images, etc; this is handled by the Jinja2 template engine.

__init__.py file

```
from flask import Flask
app = Flask(__name__)

if __name__ == '__main__':
    app.run(debug = True)
```

The above code will create an instance (*app*) for our app using the Flask class. The *app.run* method is used to run the local server with our application; we have imported the app object via *manage.py*, so we can run the server from there.

debug = True will automatically restart and reload the server if any modifications are done to any file of the application.

The *app.run* method is not actually needed here, because we are going to import *app* instance in the *manage.py* file and we can run the server from the *manage* file.

manage.py file

```
from flask import Flask, url_for, render_template
from flask.ext.script import Manager, Server

import os, sys
sys.path.append(os.path.abspath(os.path.join(os.path.
dirname('__file__'), '..')))
```